HOSTEL MANAGEMENT SYSTEM

Project Report

---

1. Project Title

Hostel Management System Using Spring Boot and MongoDB

---

2. Objective

The Hostel Management System is a backend-based web application developed using Spring Boot and MongoDB. The system is designed to manage hostel-related operations such as student records, room allocation, hostel information, staff details, fee management, visitor records, departments, and complaints.

The application provides REST APIs that can be tested using Postman and integrated with any frontend application in the future.

---

3. Technologies Used

Technology	Version
Java	17
Spring Boot	3.x
MongoDB	6.x / Latest
Maven	3.9+
Spring Data MongoDB	Latest
IntelliJ IDEA	2024+
Postman	Latest
Swagger OpenAPI	Latest
Git & GitHub	Latest

---

4. System Architecture

The application follows a layered architecture:

Controller Layer

↓

Service Layer

↓

Repository Layer

↓

MongoDB Database

Components

Controller Layer

Handles HTTP requests and responses.

Service Layer

Contains business logic.

Repository Layer

Performs database operations using Spring Data MongoDB.

Model Layer

Contains entity classes.

Database

MongoDB stores hostel management records.

---

5. Modules Implemented

The project consists of the following modules:

Student Management

- Add Student
- View Students

- Update Student
- Delete Student

Hostel Management

- Add Hostel
- View Hostel Details
- Update Hostel
- Delete Hostel

Room Management

- Manage Room Information
- Room Availability Tracking

Staff Management

- Add Staff
- Update Staff
- Delete Staff

Fee Management

- Fee Collection Records
- Payment Tracking

Department Management

- Manage Departments

Visitor Management

- Visitor Entry Records

Complaint Management

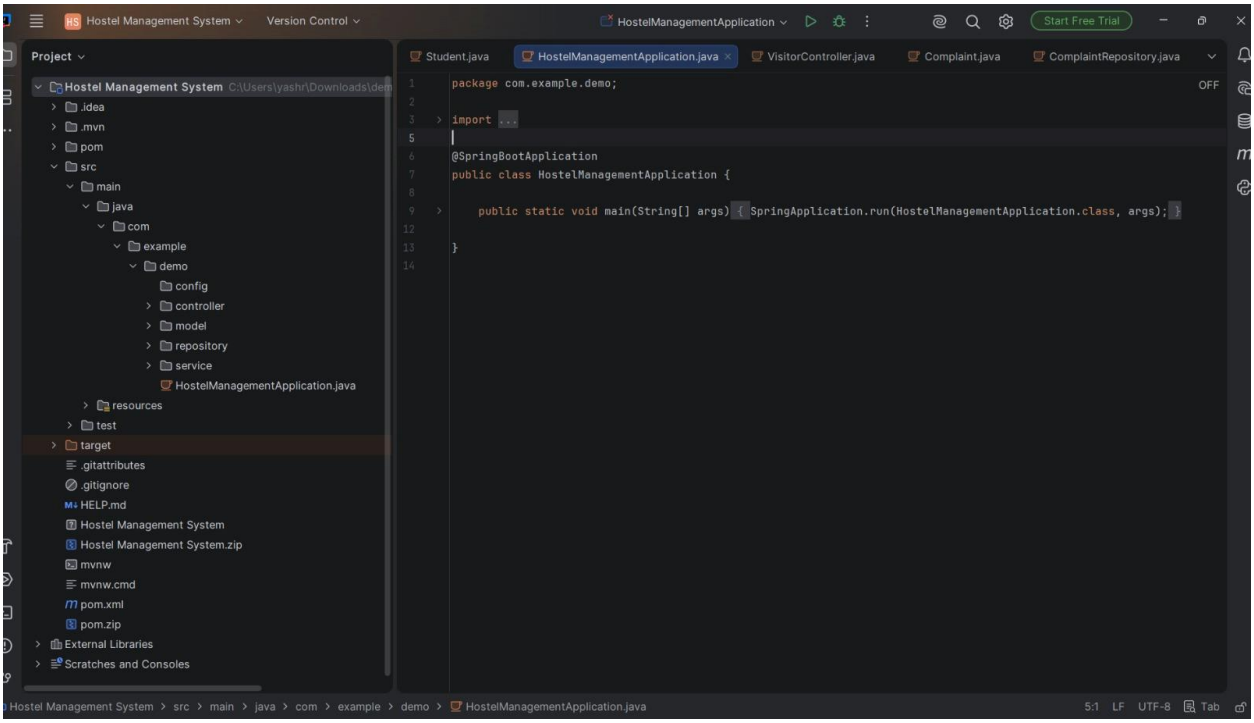
- Register Complaints
- Track Complaints

---

6. Project Structure

Screenshot 1: Complete Project Structure

Explanation



This screenshot demonstrates the complete Spring Boot project architecture.

The project contains:

Controller Package

- ComplaintController
- DepartmentController
- FeeController
- HostelController
- RoomController
- StaffController
- StudentController
- VisitorController

These classes expose REST APIs.

Model Package

- Complaint
- Department
- Fee
- Hostel
- Room
- Staff
- Student
- Visitor

These classes represent MongoDB documents.

Repository Package

- ComplaintRepository
- DepartmentRepository
- FeeRepository
- HostelRepository
- RoomRepository
- StaffRepository
- StudentRepository
- VisitorRepository

Repositories provide database CRUD operations.

Service Package

- ComplaintService
- DepartmentService
- FeeService

- HostelService
- RoomService
- StaffService
- StudentService
- VisitorService

Services implement business logic.

Resources

Contains:

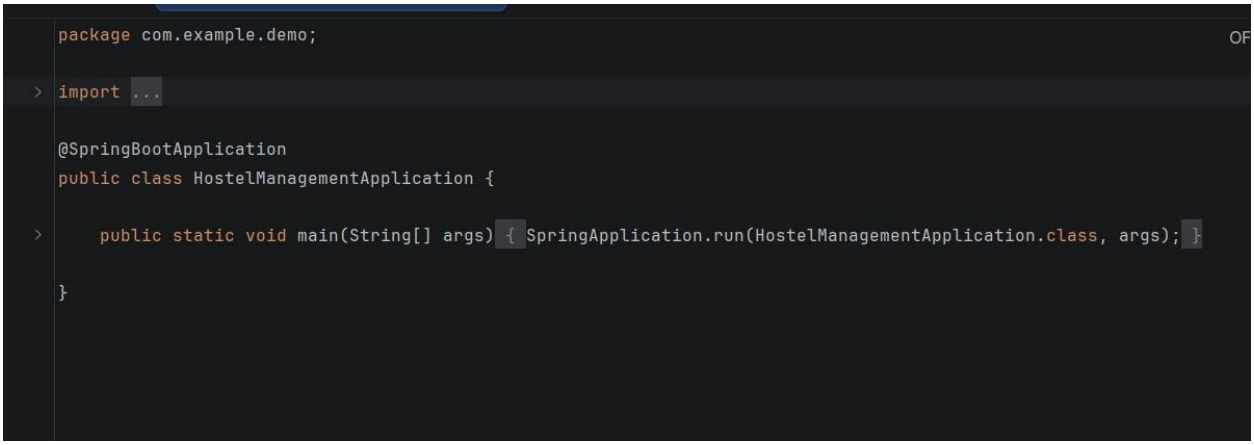
application.properties

which stores MongoDB configuration.

---

7. Main Application Class

Screenshot 2: HostelManagementApplication.java



Explanation

The application starts from the following class:

@SpringBootApplication

public class HostelManagementApplication {

```
    public static void main(String[] args) {
        SpringApplication.run(
            HostelManagementApplication.class,
            args
        );
    }
}
```

Purpose

- Bootstraps Spring Boot application.
- Performs component scanning.
- Initializes embedded server.
- Connects application with MongoDB.

---

8. MongoDB Database Configuration

Configuration

Stored inside:

src/main/resources/application.properties

Example:

```
spring.data.mongodb.uri=mongodb://localhost:27017/hosteldb
spring.data.mongodb.database=hosteldb
server.port=8080
```

Purpose

- Connect application with MongoDB.
- Configure application port.
- Configure database name.

---

9. REST APIs Implemented

Student APIs

Method	Endpoint
POST	/students
GET	/students
GET	/students/{id}
PUT	/students/{id}
DELETE	/students/{id}

Hostel APIs

Method	Endpoint
POST	/hostels
GET	/hostels
PUT	/hostels/{id}
DELETE	/hostels/{id}

Room APIs

Method	Endpoint
POST	/rooms
GET	/rooms
PUT	/rooms/{id}
DELETE	/rooms/{id}

Staff APIs

Method	Endpoint
POST	/staff
GET	/staff
PUT	/staff/{id}
DELETE	/staff/{id}

Fee APIs

Method	Endpoint
POST	/fees
GET	/fees
PUT	/fees/{id}
DELETE	/fees/{id}

Department APIs

Method	Endpoint
POST	/departments
GET	/departments
PUT	/departments/{id}
DELETE	/departments/{id}

Visitor APIs

Method	Endpoint
POST	/visitors

Method	Endpoint
GET	/visitors
PUT	/visitors/{id}
DELETE	/visitors/{id}

Complaint APIs

Method	Endpoint
POST	/complaints
GET	/complaints
PUT	/complaints/{id}
DELETE	/complaints/{id}

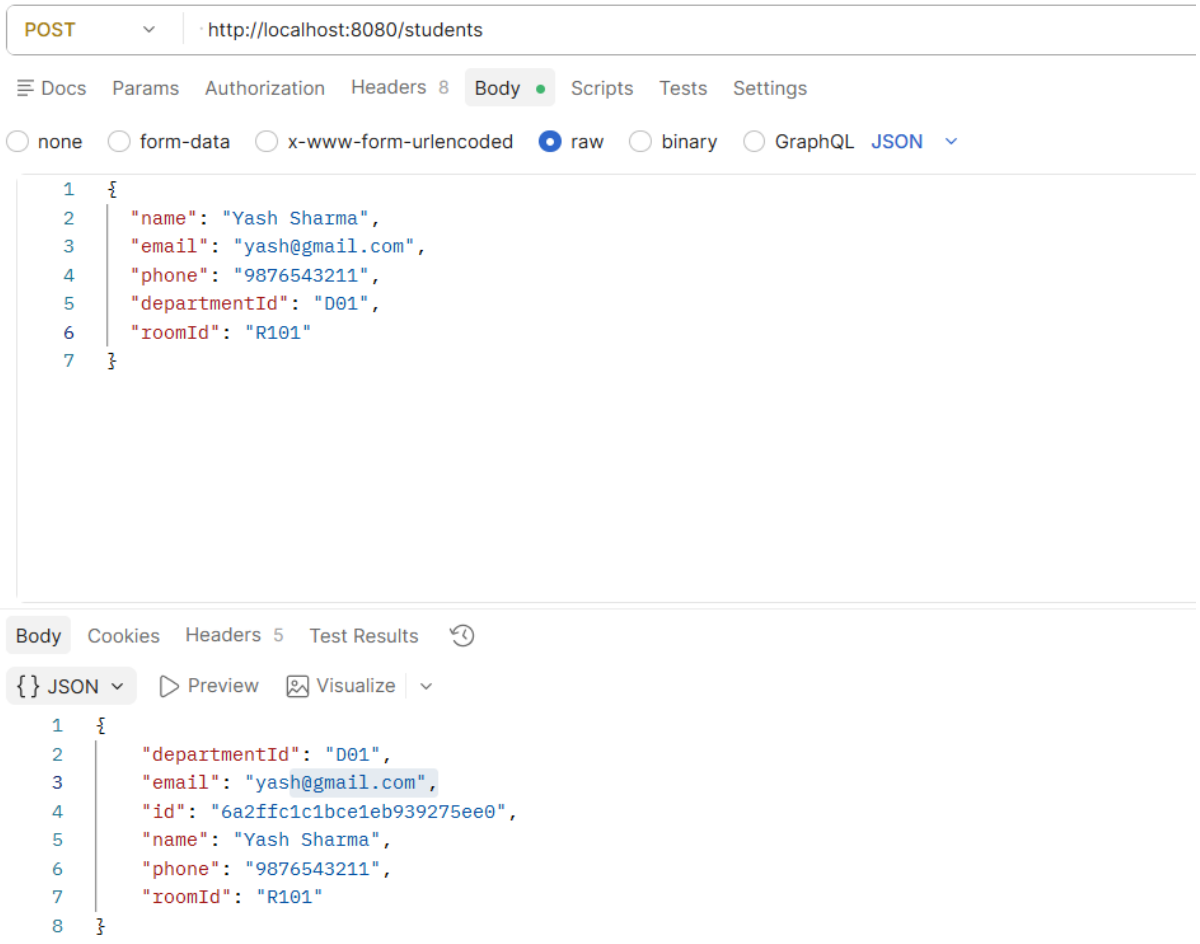
---

10. Postman Testing

For report validation, include screenshots of successful API execution.

Screenshot 3

Add Student (POST)

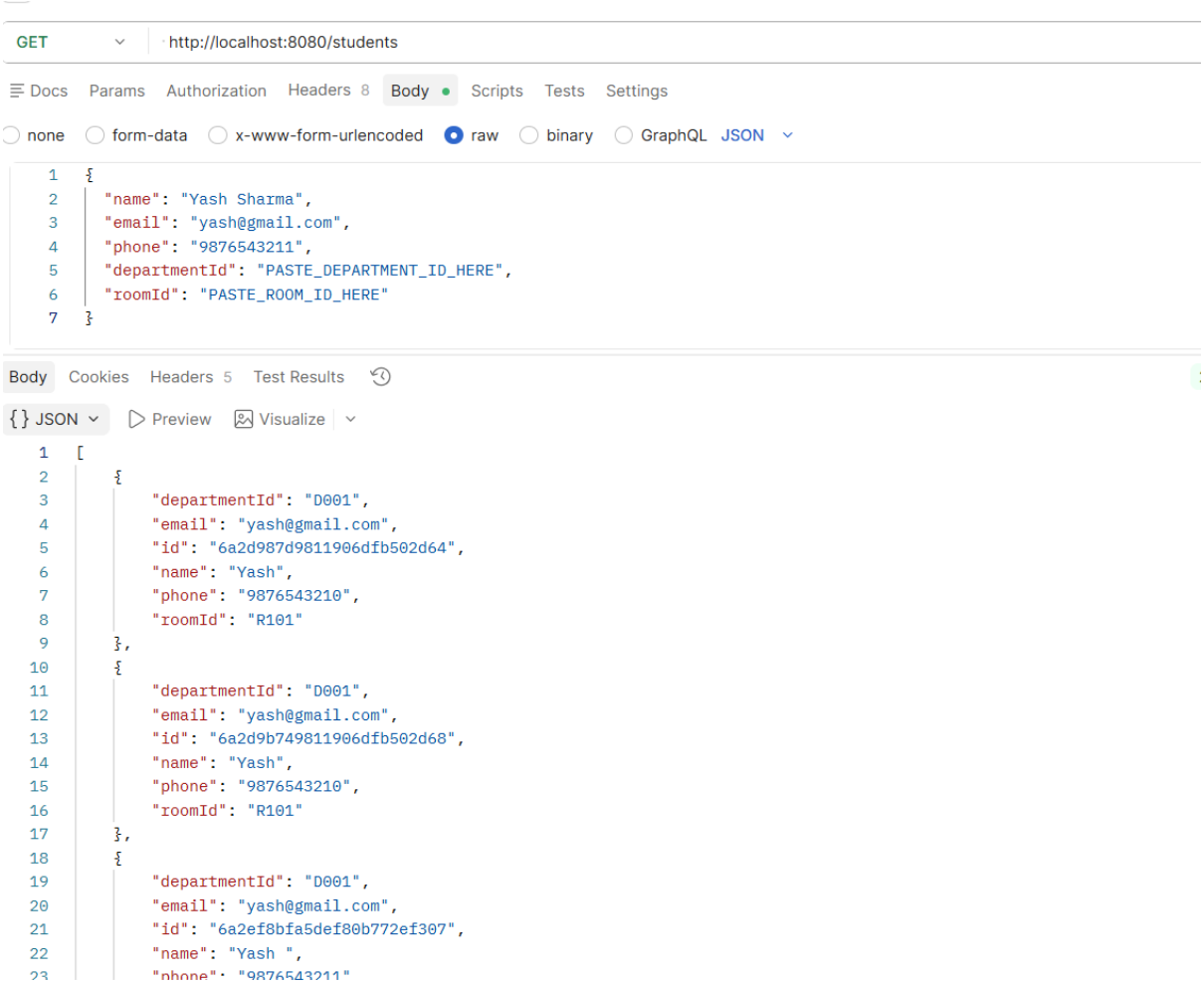


Explanation

A new student record is created and stored in MongoDB.

Screenshot 4

Get All Students (GET)

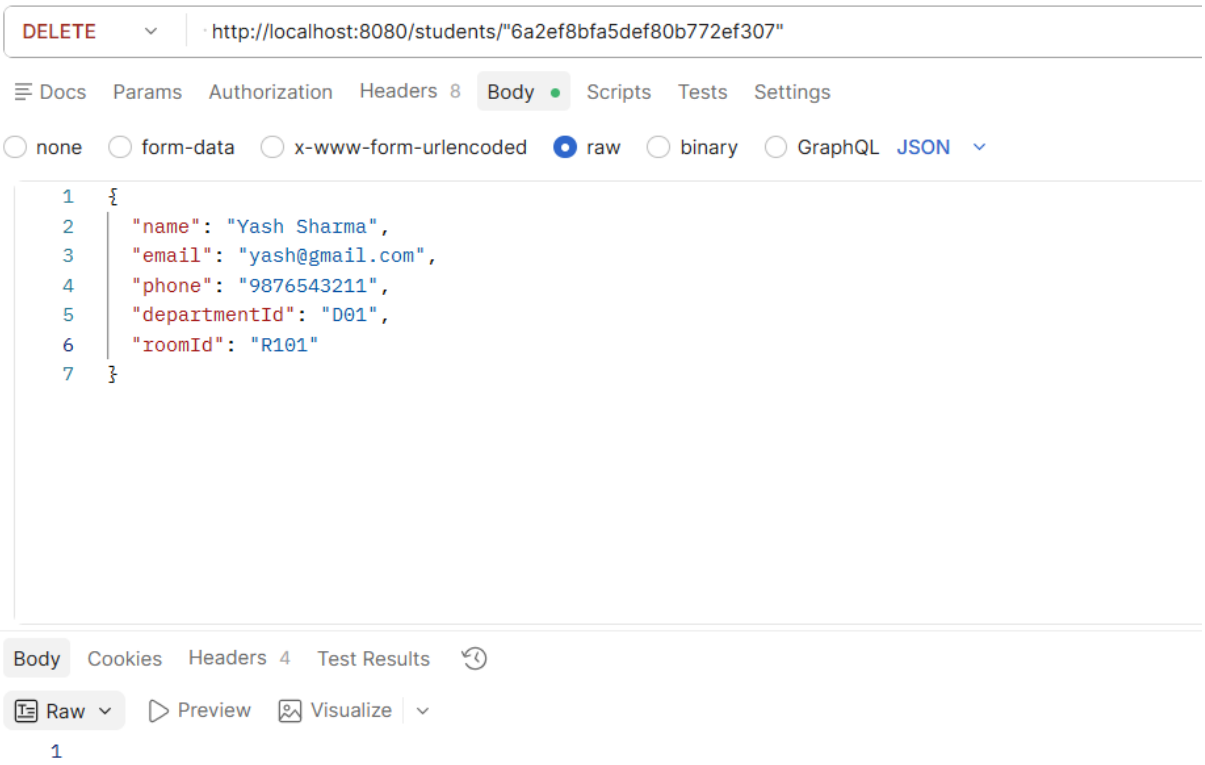


Explanation

Retrieves all student records from the database

Screenshot 5

Delete Student (DELETE)

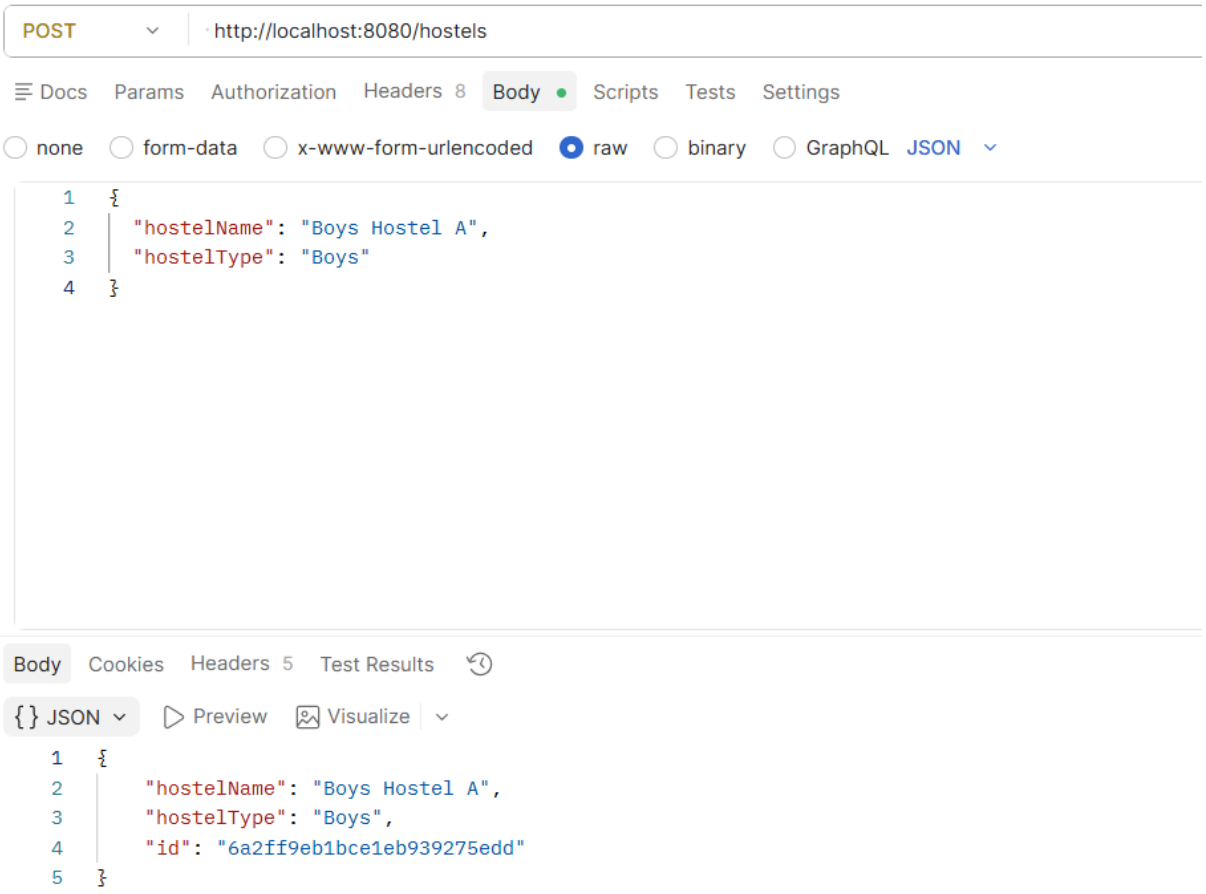


Explanation

Deletes student record successfully.

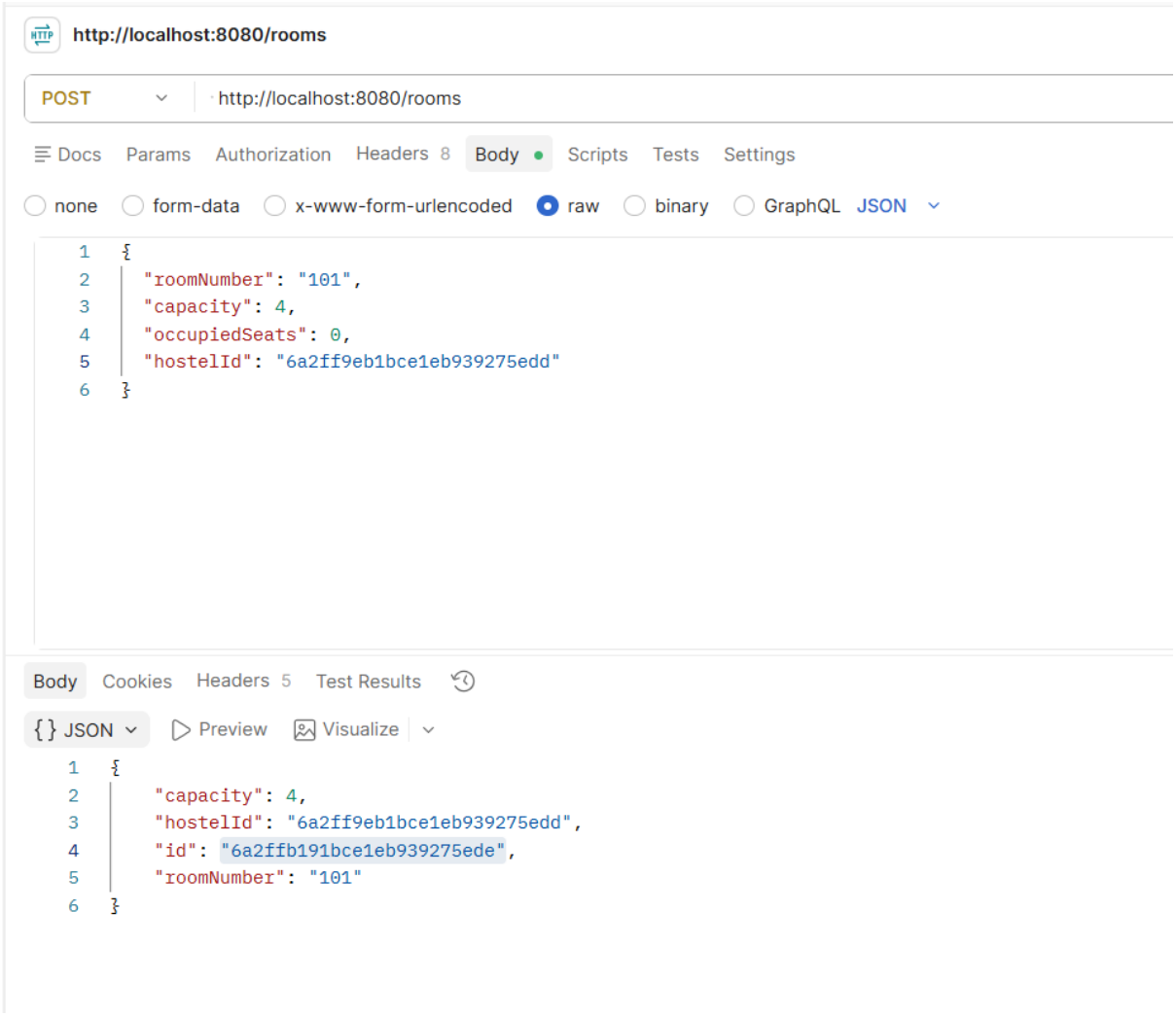
Screenshot 6

Add Hostel



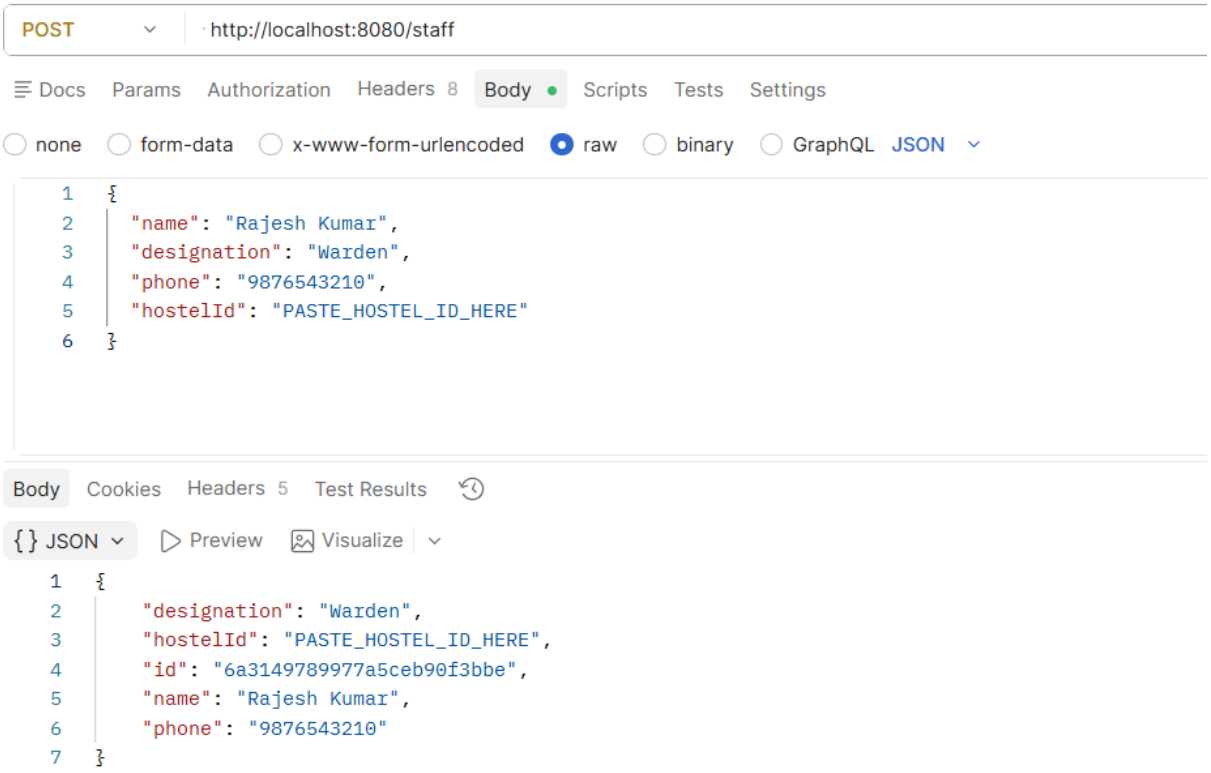
Screenshot 7

Add Room



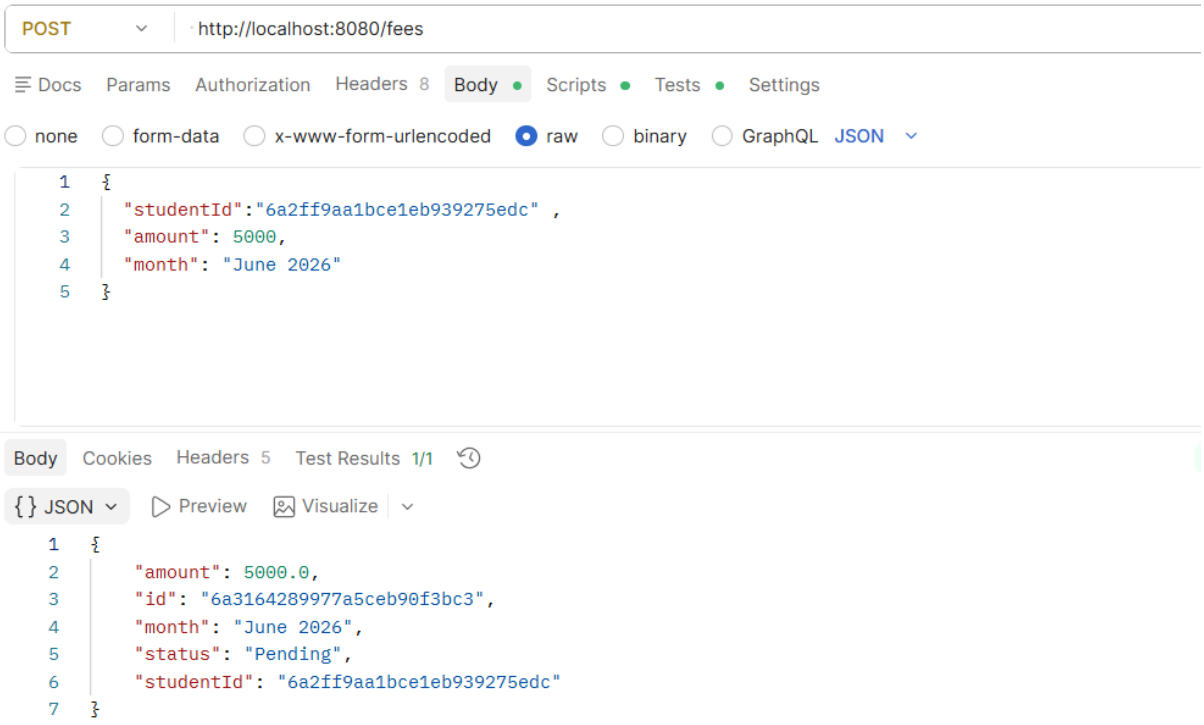
Screenshot 8

Add Staff



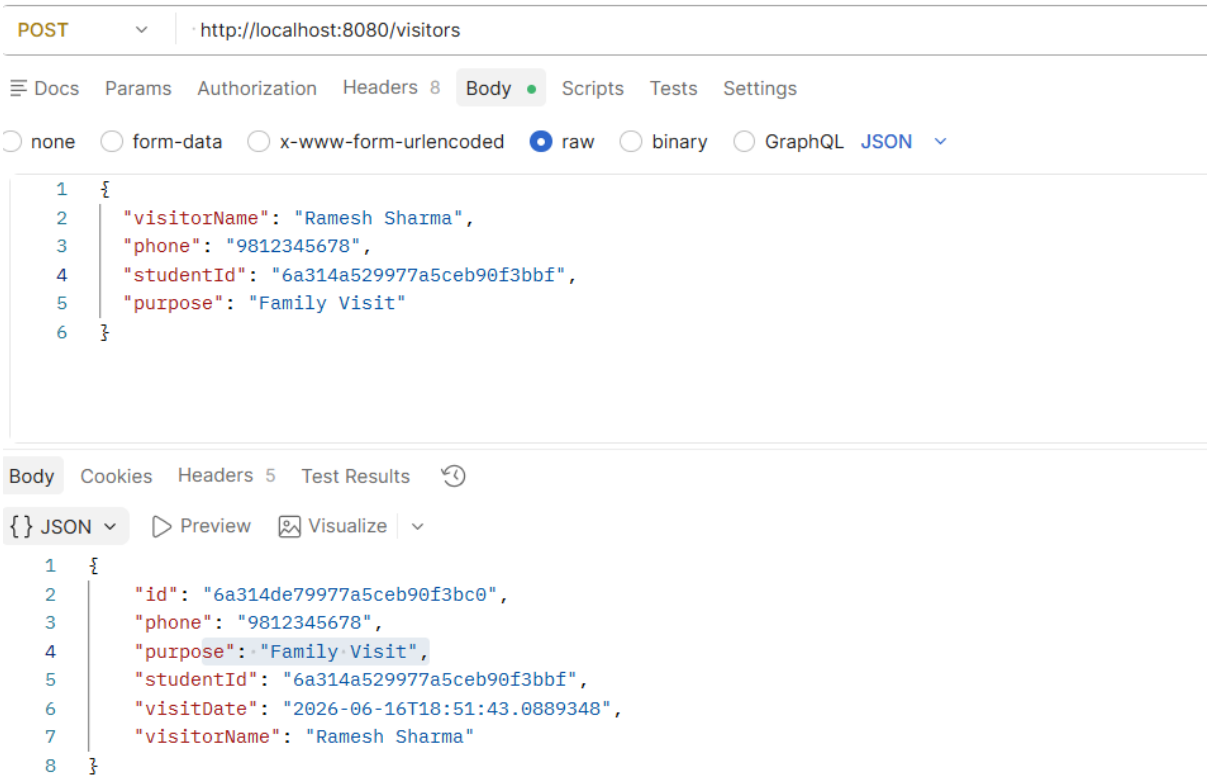
Screenshot 9

Add Fee Record



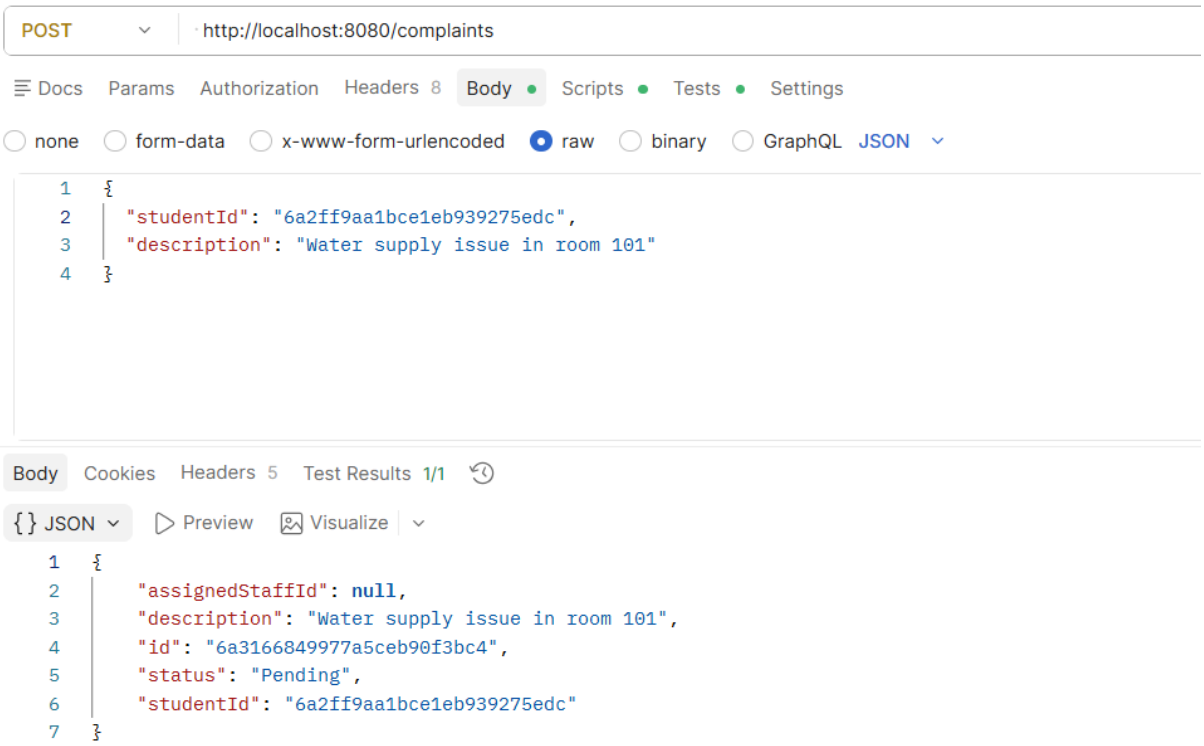
Screenshot 10

Add Visitor



Screenshot 11

Add Complaint

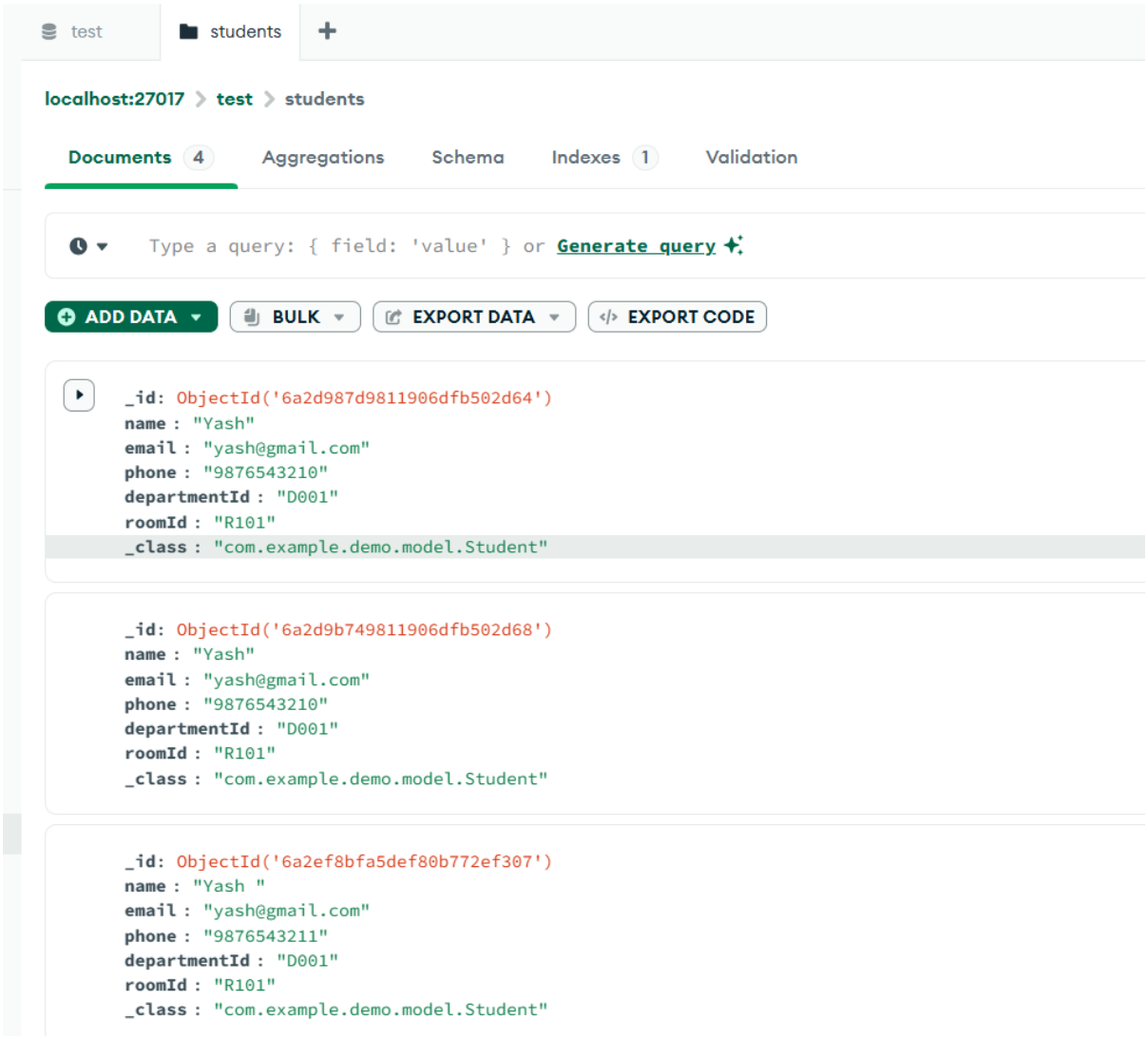


---

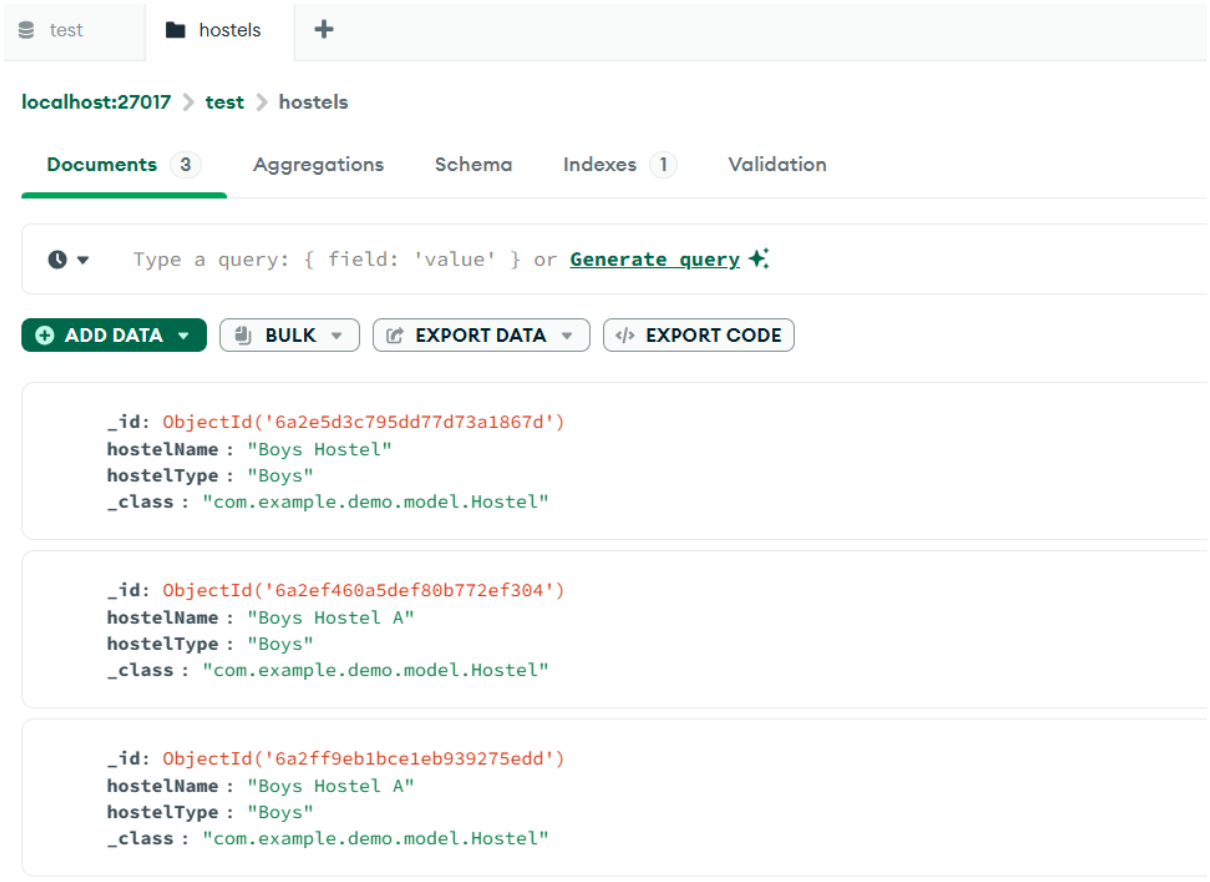
11. Output

Expected Results

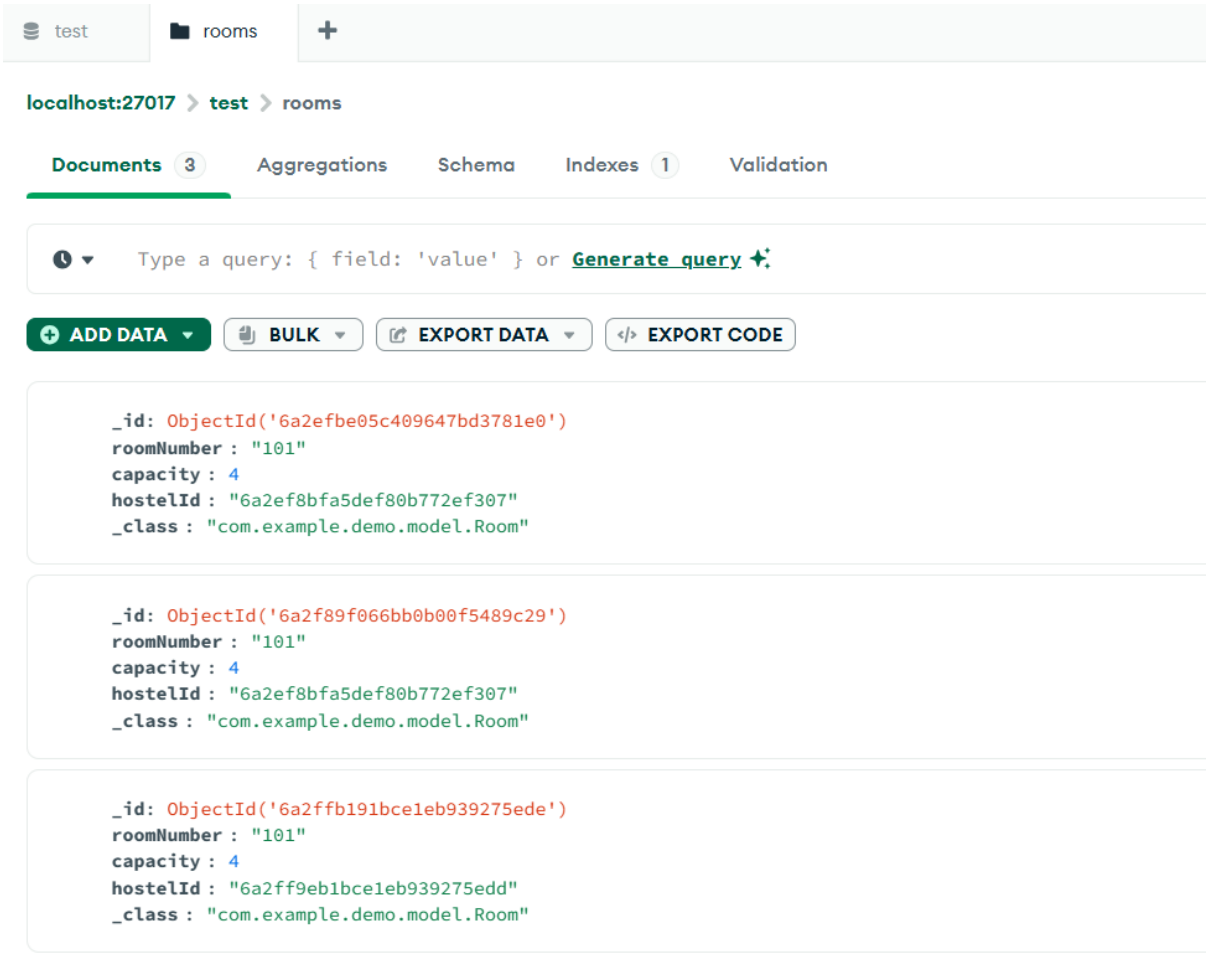
- Student records successfully managed.



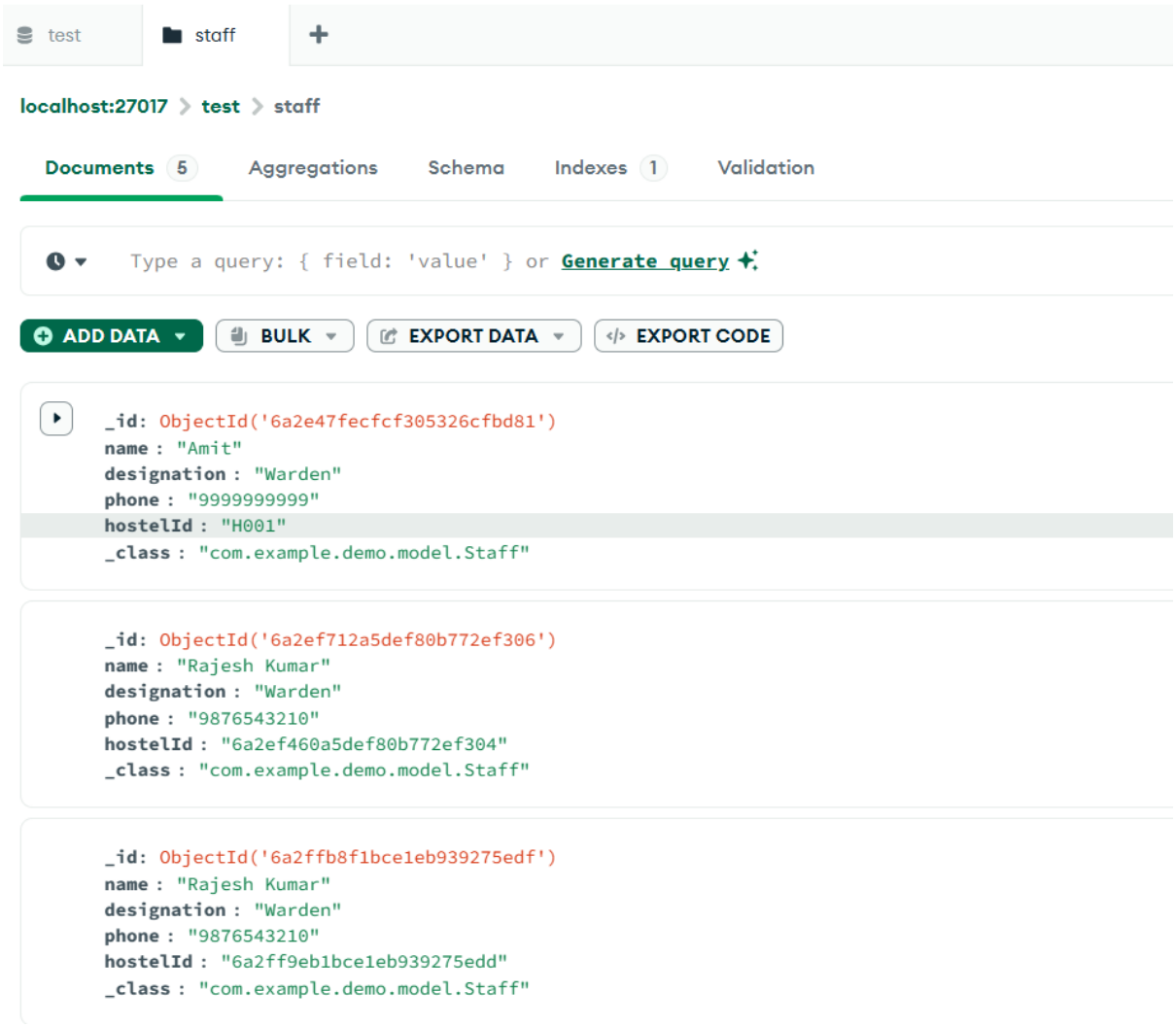
- Hostel information maintained.



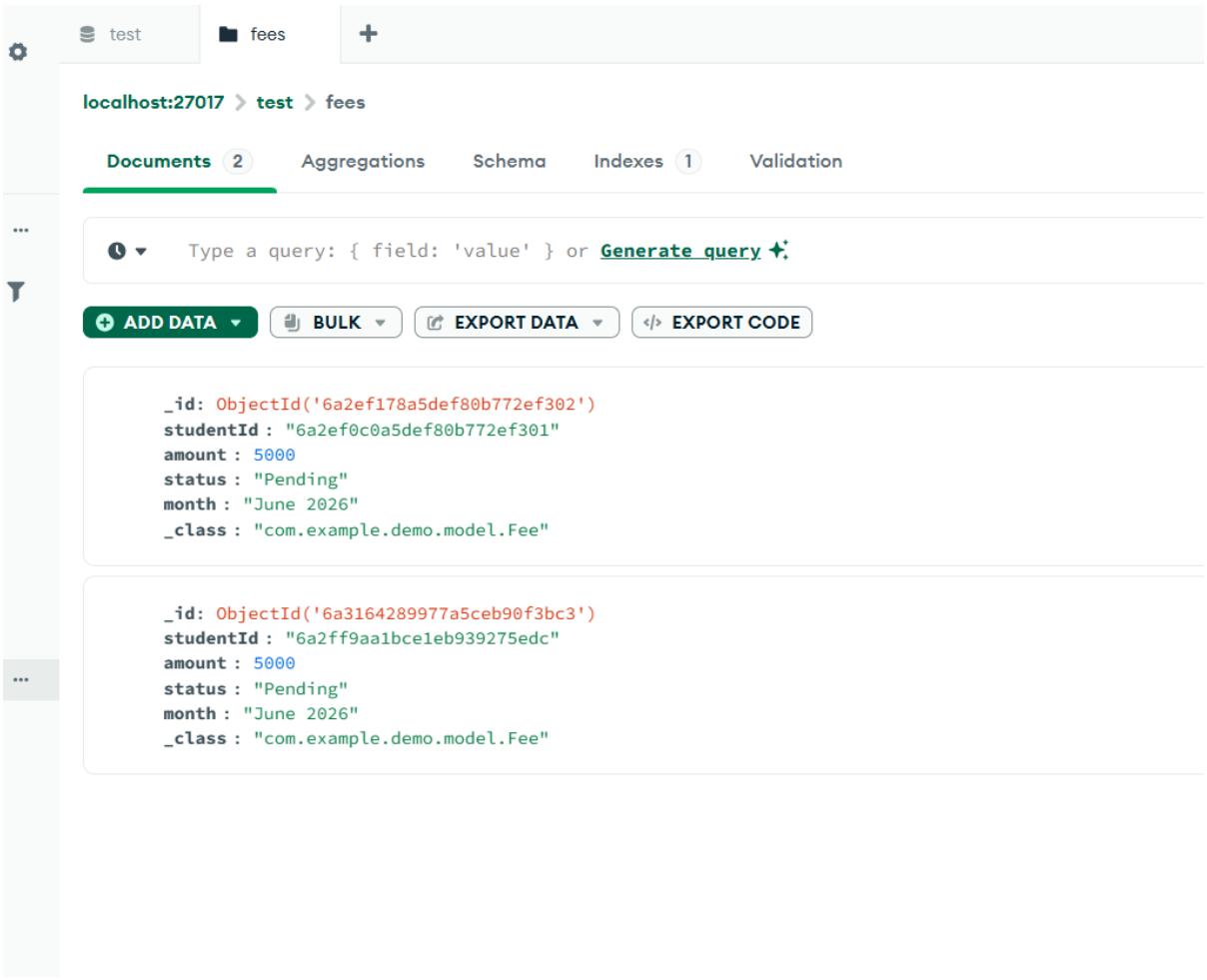
- Room allocations stored.



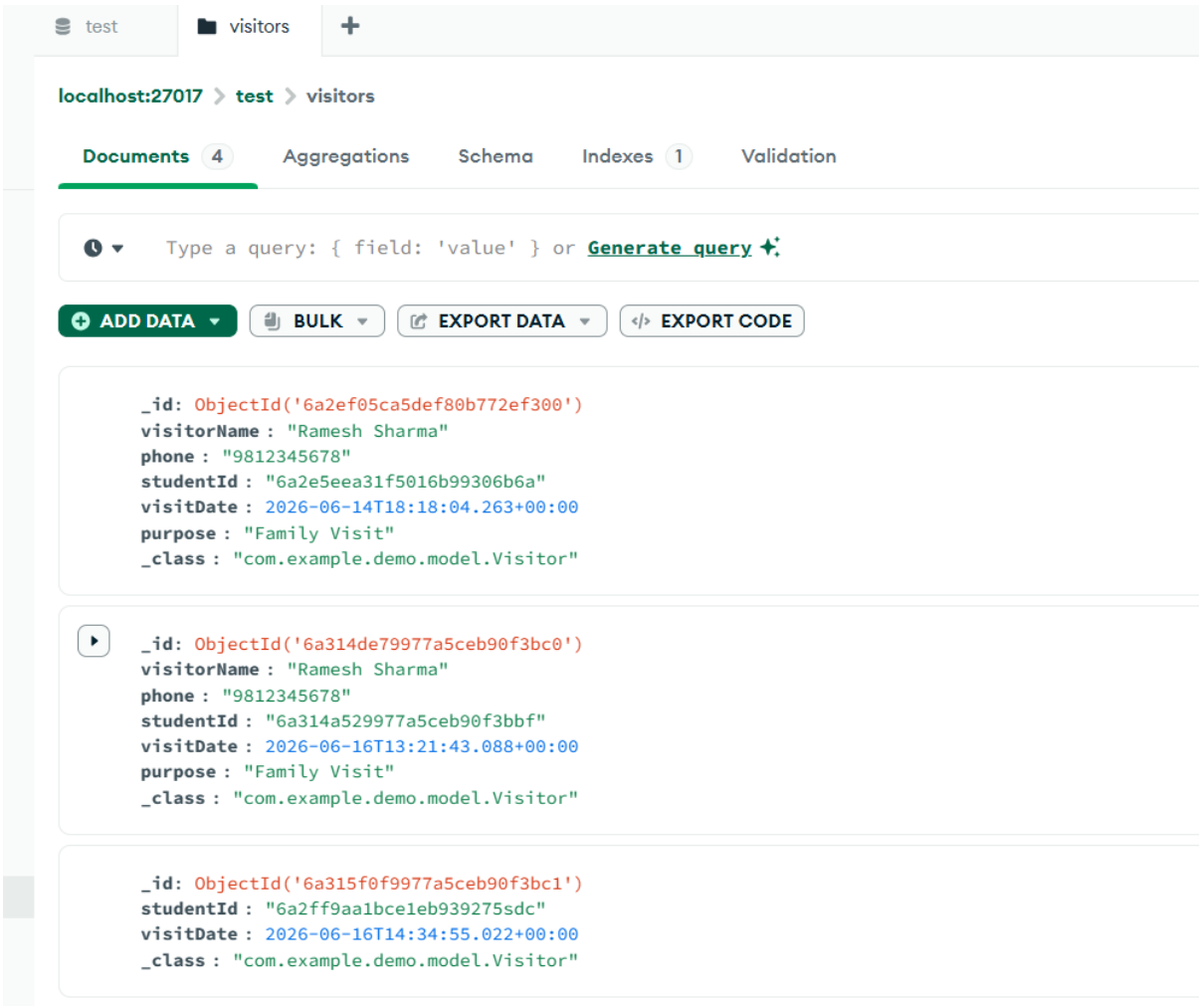
- Staff information tracked.



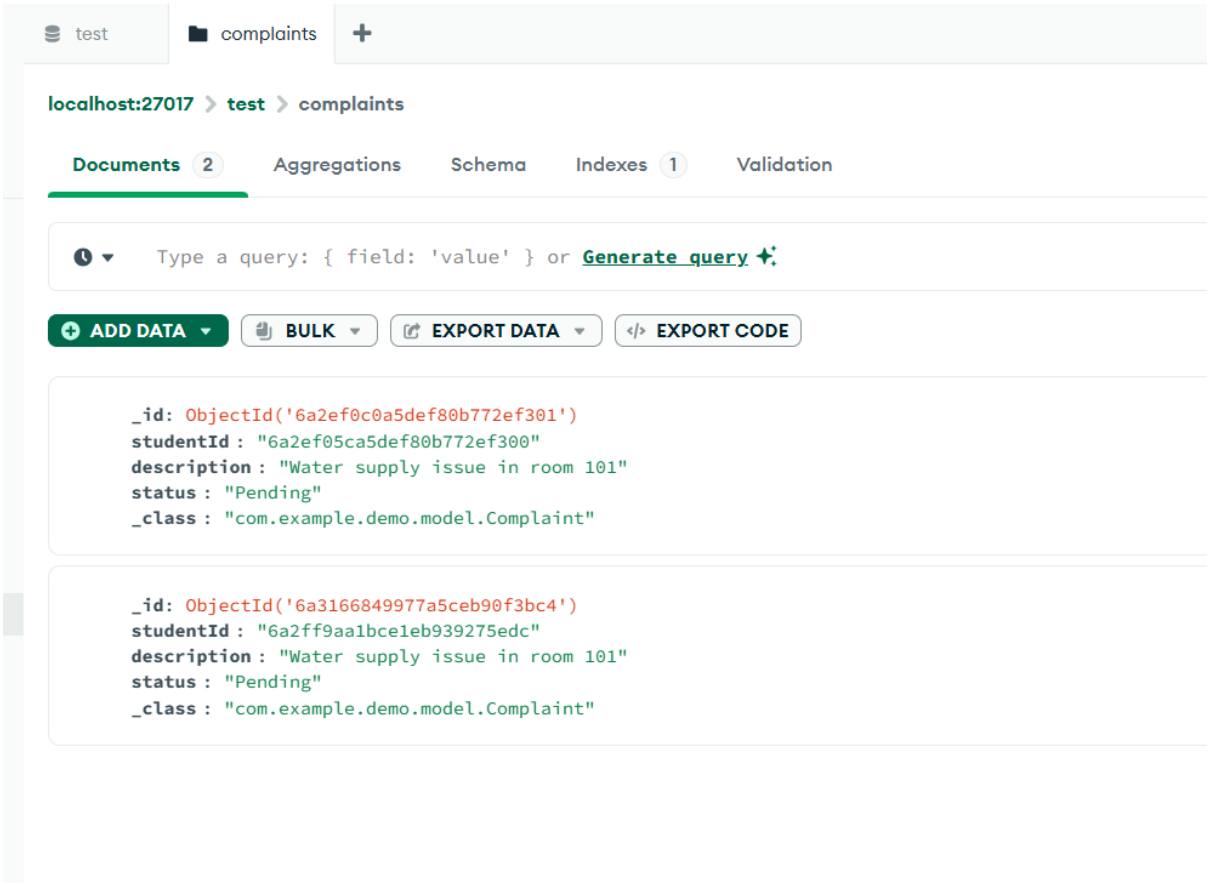
- Fee records managed.



- Visitor entries recorded.



- Complaints tracked and resolved.



---

12. Outcome

Successfully developed a complete backend-based Hostel Management System using Spring Boot and MongoDB that provides RESTful APIs for managing hostel operations. The application follows a layered architecture and supports CRUD operations for all major hostel entities.

---

13. Conclusion

The Hostel Management System simplifies hostel administration by digitizing student records, room management, staff information, fee collection, visitor tracking, department management, and complaint handling. Spring Boot provides a scalable backend framework while MongoDB offers flexible and efficient data storage. The system is fully testable using Postman and documented through Swagger, making it suitable for academic projects as well as future real-world extensions.